

Rendu individuel

Anis Douadi

Orchestration Kubernetes / Helm & on-premise / PRA

Auteur	Anis Douadi
Rôle	Orchestration Kubernetes / Helm & on-premise / PRA
Classe / Promo	M2 DO C — 2025-2026
Période	Janvier – Juin 2026
Projet	Plateforme de gestion d'un cabinet d'audioprothèse
Domaine	Chart Helm, autoscaling, site on-premise, MinIO chiffré, Ansible

SUP DE VINCI — Expert en Systèmes d'Information — Mastère DevOps — Classe **M2 DO C** — Promotion 2025-2026. Document réalisé dans le cadre du projet d'étude de fin d'année.

1. Contexte et rôle dans le projet

J'ai été responsable de l'orchestration des conteneurs sur Kubernetes et du volet hybride du projet, c'est-à-dire le site on-premise et le plan de reprise d'activité. Ces sujets touchent directement à la résilience et à la pérennité des données : ce sont eux qui déterminent la capacité de la plateforme à survivre à une panne et à protéger les informations sensibles du cabinet.

Mon travail a consisté à transformer les images produites par l'équipe en un déploiement Kubernetes cohérent, résilient et paramétrable, puis à concevoir la chaîne de sauvegarde reliant le cloud à un site on-premise simulé. J'ai donc collaboré étroitement avec Zaafer sur l'application, Elyess sur l'intégration du déploiement dans la chaîne, et Adame sur les aspects de sécurité et de supervision du cluster.

2. Ma contribution détaillée

2.1 Conception du chart Helm

J'ai conçu le chart Helm qui décrit l'ensemble des objets Kubernetes de l'application : les déploiements du backend et du frontend, les services qui les exposent, l'Ingress qui route le trafic entrant, le secret contenant la chaîne de connexion, l'autoscaling et la définition de supervision. Le chart est entièrement paramétrable : le registre d'images, l'étiquette de version, le nom d'hôte ou l'activation de certaines fonctionnalités se règlent par des valeurs, ce qui permet de l'adapter à différents environnements sans en modifier le code.

Ce packaging répond à un objectif de reproductibilité et de portabilité : le même chart, avec des valeurs différentes, peut déployer l'application en local, en pré-production ou en production. J'ai veillé à ce que le déploiement soit idempotent et à ce que la mise à jour d'une version se fasse par remplacement progressif des conteneurs, sans interruption de service.



Objets Kubernetes décrits et déployés par le chart Helm de l'application.

2.2 Résilience et autoscaling

J'ai configuré des sondes de vivacité et de disponibilité qui permettent à Kubernetes de redémarrer automatiquement un conteneur défaillant et de ne router le trafic que vers les instances prêtes. J'ai mis en place un autoscaler horizontal qui ajuste le nombre de répliques du backend en fonction de la charge processeur, dans une limite haute stricte afin de préserver la maîtrise des coûts.

Cette combinaison assure la continuité de service (PCA) : nous avons vérifié en conditions réelles que la suppression manuelle d'une instance entraîne sa recreation automatique en quelques secondes, sans intervention. J'ai également défini le contexte de sécurité des conteneurs et des politiques de cloisonnement réseau, en concertation avec le responsable sécurité.

2.3 Le site on-premise simulé

Conformément au cahier des charges, qui décrit une architecture hybride mêlant cloud et serveurs on-premise pour les données sensibles, j'ai provisionné une machine virtuelle représentant un site

on-premise. Point essentiel : cette machine est placée dans un **réseau totalement séparé** de celui du cluster, sans interconnexion directe, ce qui la fait se comporter comme un site distant joint depuis le cloud — à l'image de ce que serait un vrai site relié par Internet ou par un lien sécurisé.

Ce choix de simulation nous a permis d'illustrer concrètement le principe d'une architecture hybride sans disposer d'un centre de données physique, tout en gardant la maîtrise des coûts puisque la machine peut être désactivée par une simple variable lorsque ce volet n'est pas démontré.

2.4 Stockage objet chiffré (MinIO)

Sur cette machine, j'ai installé MinIO, un stockage objet compatible avec le standard S3, au moyen de playbooks Ansible. J'ai activé le chiffrement côté serveur, de sorte que les sauvegardes soient chiffrées au repos, répondant ainsi à l'exigence de protection des données sensibles. Un espace de stockage dédié reçoit les sauvegardes de la base.

2.5 Automatisation par Ansible et plan de reprise

J'ai écrit les playbooks Ansible qui configurent la machine on-premise et automatisent les opérations de sauvegarde et de restauration. La sauvegarde réalise un export de la base et le dépose, chiffré, dans le stockage MinIO ; la restauration récupère la sauvegarde la plus récente et la réinjecte dans la base. Ansible assure ici la gestion de configuration, en complément de Terraform qui gère le provisionnement — une répartition classique et saine des responsabilités.

J'ai rendu la restauration **idempotente**, c'est-à-dire rejouable même sur une base déjà peuplée, en incluant dans l'export les instructions de remise à zéro nécessaires. C'est cette propriété qui rend le plan de reprise réellement fiable et non seulement théorique.



3. Choix techniques et justifications

Helm s'est imposé comme le gestionnaire de paquets de référence pour Kubernetes, car il permet de décrire un ensemble d'objets de façon paramétrable et versionnée, et de gérer proprement les mises à jour et les retours arrière. MinIO a été retenu pour le stockage des sauvegardes car il est léger, compatible avec l'écosystème S3 et facile à déployer sur une simple machine, ce qui convenait parfaitement à notre site on-premise simulé.

Le recours à Ansible pour la configuration de la machine distante, plutôt qu'à des scripts ad hoc, garantit que cette configuration est déclarative, rejouable et documentée. Cette combinaison Terraform (provisionnement) + Ansible (configuration) est une pratique éprouvée qui sépare clairement la création des ressources de leur paramétrage.

4. Difficultés rencontrées et solutions apportées

La difficulté centrale a été d'orchestrer une réplication cloud vers on-premise à la fois fiable et rejouable. La première version de la restauration échouait sur une base déjà peuplée, en raison de conflits ; je l'ai corrigée en rendant l'export auto-suffisant, capable de réinitialiser les objets avant de les recréer. Ce détail, en apparence mineur, fait toute la différence entre un plan de reprise qui fonctionne en démonstration et un plan réellement exploitable.

J'ai également dû fiabiliser la connexion automatisée à la machine distante : la configuration Ansible devait attendre que la machine soit prête à accepter les connexions avant de la configurer. En coordination avec le responsable de la chaîne d'intégration, nous avons ajouté une temporisation d'attente qui garantit la robustesse de cette étape lorsqu'elle est exécutée automatiquement.

5. Perspectives d'évolution

La première évolution consisterait à activer un module réseau appliquant réellement les politiques de cloisonnement que nous avons préparées, afin d'obtenir une isolation effective entre les composants du cluster. Je viserais ensuite une haute disponibilité répartie sur plusieurs zones, ainsi qu'une réplication géographique des sauvegardes pour se prémunir contre la perte d'un site entier.

Je souhaiterais également planifier des tests de restauration réguliers et automatiques, afin de vérifier en continu l'efficacité du plan de reprise, et externaliser les sauvegardes vers un stockage redondant. Enfin, remplacer le site on-premise simulé par une véritable intégration réseau, reliée par un lien sécurisé de type VPN, rapprocherait encore la maquette d'une architecture de production.

6. Analyse critique des limites

Le site on-premise est ici simulé par une machine virtuelle placée dans un réseau distinct : l'isolation est représentative d'un site séparé, mais il ne s'agit pas d'un véritable centre de données physique, et l'interconnexion réelle par VPN n'a pas été mise en œuvre. De plus, la restauration ramène l'état au moment de la dernière sauvegarde, si bien que la perte de données potentielle dépend directement de la fréquence des sauvegardes (objectif de point de reprise).

Par ailleurs, le stockage MinIO fonctionne en instance unique, sans redondance interne, ce qui constituerait un point de défaillance en production ; et l'architecture réseau retenue pour le cluster, choisie pour sa légèreté, limite l'application stricte des politiques de cloisonnement que j'ai définies.

7. Annexe — documentation utilisateur (mon périmètre)

7.1 Consulter le stockage de sauvegarde

La console web de MinIO, accessible sur la machine on-premise, permet de visualiser l'espace de stockage dédié et d'y retrouver les sauvegardes chiffrées, chacune horodatée. C'est le moyen le plus simple de vérifier qu'une sauvegarde a bien été produite.

7.2 Déclencher une sauvegarde ou une restauration

Les opérations de sauvegarde et de restauration se déclenchent depuis le workflow prévu à cet effet, en choisissant le mode souhaité. La sauvegarde est par ailleurs planifiée quotidiennement de façon automatique. La restauration récupère et réinjecte la sauvegarde la plus récente.

7.3 Adapter le déploiement

Le chart Helm étant paramétrable, il est possible d'ajuster le registre d'images, le nom d'hôte d'accès ou l'activation de certaines fonctionnalités par de simples valeurs, sans modifier le code du chart, ce qui facilite l'adaptation à un nouvel environnement.

8. Analyse personnelle

Défis rencontrés

Mon principal défi a été de concevoir une réplication cloud vers on-premise fiable et rejouable, et de rendre la restauration réellement idempotente. Il m'a fallu comprendre en profondeur le comportement de l'outil de sauvegarde pour transformer une procédure fragile en un plan de reprise digne de confiance.

Forces

Je retiens ma maîtrise du packaging Kubernetes et des objets qui le composent, ainsi qu'une vision claire des enjeux de résilience et d'architecture hybride, qui m'a permis de relier des sujets souvent traités séparément (orchestration, réseau, sauvegarde).

Faiblesses

Je dois passer d'un on-premise simulé à une intégration réseau plus réaliste, ce qui suppose d'approfondir mes compétences en interconnexion réseau (VPN, routage). Par ailleurs, la redondance du stockage de sauvegarde reste à renforcer.

Compétences développées

Ce projet a consolidé mes compétences sur Kubernetes et Helm (templating, autoscaling, rôles, politiques réseau), sur Ansible (playbooks, gestion de configuration) et sur la conception d'un plan de reprise d'activité complet, de la sauvegarde chiffrée à la restauration automatisée.

Axes d'amélioration personnels

Je souhaite mettre en œuvre une haute disponibilité multi-zones, automatiser des tests de restauration réguliers pour garantir le plan de reprise dans la durée, et me former à l'interconnexion sécurisée de sites afin de concevoir de véritables architectures hybrides.